

Explain backtracking code

Yes. This is a **backtracking** solution for **Letter Case Permutation**. The key thing to understand is that at every letter, we have **two choices**:

1. Keep the current case.
2. Toggle the case.

For digits, there is only **one choice**.

Let's break your exact code down.

1. What problem are we solving?

For:

```
s = "a1b2"
```

We need all possible strings by changing the case of letters:

```
a1b2
a1B2
A1b2
A1B2
```

The 1 and 2 cannot change.

2. Understand the function

```
void backtrack(string &s, int i, vector<string> &res)
```

There are three important things:

```
s    → current string
i    → which character we are currently deciding
res  → final answers
```

For example:

```
s = "a1b2"
    ↑
    i
```

If:

```
i = 0
```

we are deciding what to do with 'a'.

If:

```
i = 1
```

we are deciding what to do with '1'.

3. Base case

```
if (i == s.size()) {
    res.push_back(s);
    return;
}
```

Suppose:

```
s = "a1b2"
```

When:

```
i = 4
```

we have reached the end:

```
a 1 b 2
-----
0 1 2 3
```

So:

```
i == s.size()
```

means:

We have made a decision for every character.

Therefore, store the current string:

```
res.push_back(s);
```

4. The most important line

```
backtrack(s, i + 1, res);
```

This happens **before** toggling.

It means:

First, keep the current character exactly as it is and move to the next character.

Suppose:

```
s = "a1b2"  
i = 0
```

Current character:

```
'a'
```

We first choose:

```
a
```

without changing it.

So:

```
backtrack(s, 1, res);
```

Now we're here:

```
a 1 b 2  
  ↑  
  i=1
```

5. What happens when we reach a letter?

We have:

```
if (isalpha(s[i])) {
```

`isalpha()` checks whether the character is a letter.

For example:

```
isalpha('a') // true  
isalpha('B') // true  
isalpha('1') // false
```

So only letters create two choices.

6. The strange-looking line

```
s[i] ^= (1 << 5);
```

This toggles the case of an ASCII letter.

Let's understand it.

ASCII values:

```
'A' = 65
'a' = 97
```

Binary:

```
65 = 01000001
97 = 01100001
```

The difference is bit 5:

```
00100000
```

And:

```
1 << 5
```

means:

```
00100000
```

Then:

```
^
```

is XOR.

So:

```
s[i] ^= (1 << 5);
```

flips that bit.

Therefore:

```
'a' → 'A'
'A' → 'a'
```

You can think of it simply as:

```
toggleCase(s[i]);
```

Conceptually.

7. Why is this backtracking?

Look at this:

```
backtrack(s, i + 1, res);
```

First choice:

```
Don't change the letter.
```

Then:

```
s[i] ^= (1 << 5);
backtrack(s, i + 1, res);
```

Second choice:

```
Change the letter's case.
```

So every letter creates a binary decision:

```
      letter
     /    \
  original toggled
```

That's the entire idea.

8. Full dry run

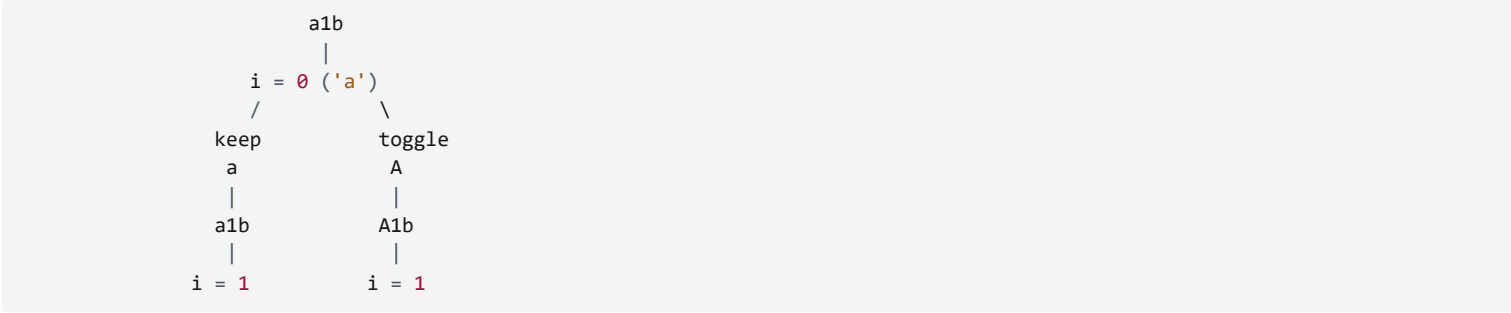
Let's use:

```
s = "a1b"
```

Initially:

```
"a1b"  
i=0
```

Tree:



Let's follow the **left side** first.

Step 1

```
backtrack("a1b", 0)
```

Current:

```
a1b  
^  
i=0
```

Execute:

```
backtrack(s, i + 1, res);
```

So:

```
backtrack("a1b", 1)
```

Step 2

Current:

```
a1b  
^  
i=1
```

'1' is a digit.

So:

```
backtrack(s, i + 1, res);
```

moves directly to:

```
a1b  
^  
i=2
```

No second branch because:

```
isalpha('1') == false
```

Step 3

Current:

```
a1b  
^  
i=2
```

Current character is 'b'.

Again:

```
backtrack(s, i + 1, res);
```

First choice:

```
keep b
```

So:

```
a1b
  ^
  i=3
```

Now:

```
i == s.size()
```

Therefore:

```
res.push_back("a1b");
```

First answer:

```
a1b
```

9. Now comes the backtracking part

We return from:

```
backtrack(s, 3, res);
```

back to:

```
i = 2
```

The string is still:

```
a1b
```

Then this executes:

```
s[i] ^= (1 << 5);
```

So:

```
b → B
```

Now:

```
a1B
```

Then:

```
backtrack(s, i + 1, res);
```

So we reach:

```
i = 3
```

and add:

```
a1B
```

Now we have:

```
res = {
  "a1b",
  "a1B"
}
```

10. What about the a?

We return back to:

```
i = 0
```

The string currently has:

```
a1B
```

Then:

```
s[i] ^= (1 << 5);
```

toggles:

```
a → A
```

So now:

```
A1B
```

Then recursion continues.

But notice something important:

We did not explicitly change `b` back from `B` to `b`.

Why does this still work?

Because the recursive calls themselves will toggle the characters again as necessary.

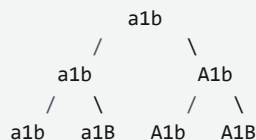
This is a subtle part of this implementation.

11. The recursion tree conceptually

For:

```
"a1b"
```

the decision tree is:



At the leaves:

```
a1b
a1B
A1b
A1B
```

These are exactly the answers.

12. But there is something you should notice about this code

Your code is clever but slightly confusing because it **mutates the string** without explicitly undoing the toggle.

A more standard backtracking implementation is:

```
void backtrack(string &s, int i, vector<string>& res) {

    if (i == s.size()) {
        res.push_back(s);
        return;
    }

    // Choice 1: keep current character
    backtrack(s, i + 1, res);

    // Choice 2: change case
    if (isalpha(s[i])) {
        s[i] ^= (1 << 5);
        backtrack(s, i + 1, res);

        // Undo the change
        s[i] ^= (1 << 5);
    }
}
```

The last line:

```
s[i] ^= (1 << 5);
```

is the **backtracking/undo** step.

This makes the state explicit:

```
choose
↓
explore
↓
undo
```

13. The most important mental model

Don't think of this code as:

"Change every character somehow."

Think:

At every letter, create two branches. At every digit, create one branch.

For:

a1b2

you get:



Therefore:

2 x 2 = 4

answers.

If there are k letters:

number of answers = 2^k

The three lines you should understand most

```
backtrack(s, i + 1, res);
```

Choice 1: keep the current character.

```
s[i] ^= (1 << 5);
```

Choice 2: toggle the current character.

```
backtrack(s, i + 1, res);
```

Explore Choice 2.

That's the entire backtracking idea behind this solution.